



Instituto Tecnológico de Costa Rica

Campus Tecnológico Central Cartago

Escuela de Ingeniería en Computación

TravelSplit: Base de datos

IC-6821 Diseño de Software GR1

Prof. Ing. Mario Chacón Rivas

Cesar Gamboa Naranjo - 2024138712

Daniel Morera Elizondo - 2024114502

I Semestre

Año 2026

Índice

1. Introducción	3
2. Estándares	4
2.1 Consideraciones	4
2.2 Nombres de tablas	4
2.3 Nombres de columnas	4
2.4 Asignación de tipos de datos	5
2.5 Llaves primarias y foráneas	7
2.6 Plataforma y parámetros de BD	8
3. Modelo de datos	9
3.1 Visión general	9
3.2 Tablas	10
4. Análisis de tamaños	16
4.1 Tamaño de registro	16
4.2 Análisis del tamaño de tabla	18
4.3 Resumen general del almacenamiento proyectado	20
5. Conclusiones	22

1. Introducción

El sistema TravelSplit es una aplicación web diseñada para facilitar la gestión de gastos compartidos durante viajes, giras, salidas grupales y cualquier actividad en la que varias personas deban dividir costos. La idea principal es evitar confusiones, cálculos manuales y problemas al momento de repartir gastos, ofreciendo una herramienta clara y organizada que permita llevar un control detallado de quién pagó qué y cuánto debe cada participante.

Este documento presenta el diseño del modelo de base de datos relacional que soporta el sistema, incluyendo la definición de tablas, relaciones, tipos de datos y decisiones de diseño orientadas a la eficiencia y escalabilidad.

2. Estándares

Para garantizar la mantenibilidad, escalabilidad y legibilidad del modelo de datos, se definen los siguientes estándares y convenciones que deberán aplicarse de manera uniforme en todo el esquema de base de datos de TravelSplit.

2.1 Consideraciones

Claridad semántica: cada nombre debe describir con precisión su propósito en el dominio de TravelSplit (gestión de gastos compartidos en viajes grupales).

Consistencia: mismas reglas de nombrado y tipado en todo el esquema de base de datos.

Eficiencia y crecimiento: tipos de datos ajustados al volumen esperado de usuarios, viajes y gastos, considerando proyecciones de uso a futuro.

Integridad: restricciones (NOT NULL, UNIQUE, FOREIGN KEY, CHECK) que eviten datos huérfanos, duplicados o inválidos.

Accesibilidad y seguridad: almacenamiento de contraseñas exclusivamente mediante hash (bcrypt), datos mínimos necesarios por usuario, y metadatos de auditoría para facilitar trazabilidad.

2.2 Nombres de tablas

Idioma: español, plural y descriptivo: Usuarios, Viajes, Gastos, UsuariosViajes, DivisionGastos.

Estilo: snake_case sin tildes, espacios ni caracteres especiales.

Tablas de relación muchos a muchos: se nombran combinando las dos entidades en orden alfabético.

2.3 Nombres de columnas

Llaves primarias: id_ + nombre de la tabla en singular.

Llaves foráneas: id_ + nombre de la tabla destino en singular.

Cadenas de texto: sustantivos o adjetivos claros.

2.4 Asignación de tipos de datos

La selección adecuada de tipos de datos es fundamental para garantizar la eficiencia del almacenamiento, la integridad de la información y el rendimiento de las consultas. A continuación, en la tabla 1, se especifica el tipo de dato asignado a cada columna según la naturaleza de los datos que almacenará.

Tabla	Columna	Tipo de dato	Justificación
Usuarios	id_usuario	SERIAL	Identificador numérico autoincremental
Usuarios	nombre	VARCHAR(64)	Nombres completos.
Usuarios	correo	VARCHAR(64)	Longitud estándar para correos electrónicos
Usuarios	contrasena	VARCHAR(256)	Hash bcrypt para seguridad.
Viajes	id_viaje	SERIAL	Identificador autoincremental.
Viajes	nombre	VARCHAR(128)	Títulos de viaje razonablemente largos.

Viajes	fecha_creacion	TIMESTAMP	Registro automático del momento de creación.
UsuariosViajes	id_usuario_viaje	SERIAL	Identificador único de la relación.
UsuariosViajes	id_viaje	INT	Llaves foráneas estándar.
UsuariosViajes	balance	NUMERIC(10,2)	Valores monetarios con dos decimales.
UsuariosViajes	rol	VARCHAR(16)	Dominio pequeño: 'admin', 'participante'.
Gastos	id_gasto	SERIAL	Identificador autoincremental.
Gastos	id_viaje, id_usuario	INT	Llaves foráneas.
Gastos	monto	NUMERIC(10,2)	Monto total del gasto (>0).
Gastos	descripcion	VARCHAR(256)	Suficiente para describir un gasto.
Gastos	fecha_creacion	TIMESTAMP	Auditoría de cuándo se registró el gasto.

DivisionGastos	id_division	SERIAL	Identificador autoincremental.
DivisionGastos	id_gasto, id_usuario	INT	Llaves foráneas.
DivisionGastos	monto	NUMERIC(10,2)	Monto asignado a ese usuario (≥ 0).

Tabla 1. Tabla de tipos de datos por columna.

2.5 Llaves primarias y foráneas

Llaves primarias (PK): una por tabla, columna simple id_* de tipo SERIAL (auto incremental).

Llaves foráneas (FK):

UsuariosViajes.id_viaje $\rightarrow$ Viajes.id_viaje

UsuariosViajes.id_usuario $\rightarrow$ Usuarios.id_usuario

Gastos.id_viaje $\rightarrow$ Viajes.id_viaje

Gastos.id_usuario $\rightarrow$ Usuarios.id_usuario

DivisionGastos.id_gasto $\rightarrow$ Gastos.id_gasto

DivisionGastos.id_usuario $\rightarrow$ Usuarios.id_usuario

Restricciones adicionales:

UNIQUE (id_viaje, id_usuario) en UsuariosViajes para evitar duplicados.

UNIQUE (id_gasto, id_usuario) en DivisionGastos para evitar dividir dos veces el mismo gasto al mismo usuario.

CHECK (monto > 0) en Gastos.

CHECK (monto >= 0) en DivisionGastos.

CHECK (rol IN ('admin', 'participante')) en UsuariosViajes.

2.6 Plataforma y parámetros de BD

Motor de base de datos: PostgreSQL 15.x o superior

Juego de caracteres por defecto: UTF8.

Zona horaria: UTC para todos los TIMESTAMP.

3. Modelo de datos

El presente apartado describe la estructura del modelo de datos propuesto para el sistema TravelSplit, a continuación, se presenta una visión general del modelo, donde se explican las entidades fundamentales del sistema y cómo interactúan entre sí dentro del contexto de la gestión de gastos compartidos en viajes grupales.

3.1 Visión general

El sistema TravelSplit se organiza alrededor de la entidad principal Viaje, ya que esta representa el núcleo funcional de la aplicación. Cada viaje agrupa a un conjunto de usuarios que participan en una actividad compartida y, a su vez, contiene los distintos gastos que se generan durante su desarrollo. De esta manera, el viaje funciona como el contenedor principal de la información relacionada con la gestión económica del grupo.

Dentro de cada viaje participan uno o varios usuarios, quienes pueden asumir distintos roles según la dinámica de uso del sistema. Algunos usuarios únicamente consultan su balance o registran gastos personales, mientras que otros pueden encargarse de administrar el viaje, agregar participantes o supervisar la distribución de los gastos. Para representar correctamente esta relación entre usuarios y viajes, se requiere una estructura intermedia que permita asociar múltiples usuarios a un mismo viaje, así como registrar su participación específica dentro de él.

Por otra parte, cada gasto pertenece a un viaje determinado y es registrado por un usuario. Los gastos constituyen una de las entidades más importantes del sistema, ya que, a partir de ellos se calcula cuánto debe pagar cada participante y cómo se distribuyen las deudas dentro del grupo. Además, un mismo gasto puede dividirse entre varios participantes, ya sea de manera equitativa o mediante montos personalizados, por lo que se hace necesario almacenar también la información correspondiente a la división de dichos gastos.

A partir de esta estructura, el sistema puede calcular balances individuales y grupales, permitiendo que cada participante conozca con claridad cuánto ha pagado, cuánto debe y cuál es su estado dentro del viaje. Este modelo de datos busca representar de forma ordenada, clara y escalable las relaciones principales del sistema, facilitando tanto la gestión de la información como su futura implementación en una base de datos relacional.

3.2 Tablas

Las tablas necesarias para llevar a cabo el proyecto, las cuales se detallan en los seccionales 3.2.n correspondientemente, son las siguientes: Usuarios, Viajes, UsuariosViajes, Gastos y DivisionGastos.

3.2.1 Usuarios

Columnas

id_usuario (PK, SERIAL)

nombre (VARCHAR(64))

correo (VARCHAR(64) UNIQUE)

contrasena (VARCHAR(256))

Restricciones y reglas

- a. La columna id_usuario es la llave primaria y se genera automáticamente mediante SERIAL.
- b. El campo correo debe ser único para evitar registros duplicados en el sistema.
- c. Los campos nombre, correo y contrasena no deben ser nulos, ya que son necesarios para el funcionamiento del sistema.
- d. La contraseña debe almacenarse de forma segura (por ejemplo, encriptada) para proteger la información del usuario.

Consultas frecuentes

1. Registro de un nuevo usuario.
2. Inicio de sesión (búsqueda por correo).
3. Obtener información de un usuario.

3.2.2 Viajes

Columnas

id_viaje (PK, SERIAL)

nombre (VARCHAR(128))

fecha_creacion (TIMESTAMP)

Restricciones y reglas

La columna id_viaje es la llave primaria y se genera automáticamente mediante SERIAL.

El campo nombre no debe ser nulo, ya que permite identificar cada viaje dentro del sistema.

El campo fecha_creacion no debe ser nulo, porque permite registrar el momento en que se creó el viaje.

Cada viaje representa una actividad grupal independiente dentro de la aplicación.

Consultas frecuentes

Crear un nuevo viaje.

Obtener la lista de viajes registrados.

Consultar la información de un viaje específico.

Ordenar viajes por fecha de creación.

3.2.3 UsuariosViajes

Columnas

id_usuario_viaje (PK, SERIAL)

id_viaje (FK, INT)

id_usuario (FK, INT)

balance (NUMERIC(10,2))

rol (VARCHAR(20))

Restricciones y reglas

La columna id_usuario_viaje es la llave primaria y se genera automáticamente mediante SERIAL.

La columna id_viaje es una llave foránea que referencia la tabla Viajes.

La columna id_usuario es una llave foránea que referencia la tabla Usuarios.

Los campos id_viaje y id_usuario no deben ser nulos, ya que permiten identificar qué usuario participa en cuál viaje.

El campo balance puede almacenar el saldo individual del participante dentro del viaje.

El campo rol no debe ser nulo y define el tipo de participación del usuario dentro del viaje.

El valor de rol puede restringirse a valores como admin o participante.

Un mismo usuario puede participar en varios viajes, y un viaje puede tener varios usuarios asociados.

Se recomienda establecer una restricción UNIQUE sobre la combinación id_viaje y id_usuario para evitar que un mismo usuario sea registrado dos veces en el mismo viaje.

Consultas frecuentes

Agregar un usuario a un viaje.

Consultar los participantes de un viaje.

Consultar el balance de un participante dentro de un viaje.

Verificar si un usuario es administrador o participante en un viaje.

Eliminar un participante de un viaje.

3.2.4 Gastos

Columnas

id_gasto(PK, SERIAL)

id_viaje (FK, INT)
id_usuario (FK, INT)
monto (NUMERIC(10,2))
descripcion (VARCHAR(256))
fecha_creacion (TIMESTAMP)

Restricciones y reglas

La columna id_gastoes la llave primaria y se genera automáticamente mediante SERIAL.

La columna id_viaje es una llave foránea que referencia la tabla Viajes.

La columna id_usuario es una llave foránea que referencia la tabla Usuarios.

Los campos id_viaje, id_usuario, monto, descripcion y fecha_creacion no deben ser nulos.

El campo monto debe almacenar valores monetarios con dos decimales, por lo que se utiliza NUMERIC(10,2).

El monto debe ser mayor que cero para representar un gasto válido.

Cada gasto debe estar asociado a un viaje específico y a un usuario que lo registró.

La tabla Gastos funciona también como base para el historial de gastos de cada viaje.

Consultas frecuentes

Registrar un nuevo gasto.

Consultar todos los gastos de un viaje.

Consultar los gastos registrados por un usuario.

Ordenar el historial de gastos por fecha.

Obtener el detalle de un gasto específico.

3.2.5 DivisionGastos

Columnas

id_division(PK, SERIAL)
id_gasto(FK, INT)

id_usuario (FK, INT)

monto (NUMERIC(10,2))

Restricciones y reglas

La columna id_divisiones la llave primaria y se genera automáticamente mediante SERIAL.

La columna id_gastos es una llave foránea que referencia la tabla Gastos.

La columna id_usuario es una llave foránea que referencia la tabla Usuarios.

Los campos id_gasto, id_usuario y monto no deben ser nulos.

El campo monto debe almacenar valores monetarios con dos decimales.

El monto debe ser mayor o igual a cero.

Cada registro representa la parte de un gasto que corresponde pagar a un usuario específico.

Un mismo gasto puede dividirse entre varios usuarios, por lo que existirán múltiples registros asociados a un mismo id_gasto.

Consultas frecuentes

Registrar la división de un gasto entre participantes.

Consultar cómo se repartió un gasto específico.

Obtener cuánto debe pagar un usuario en un gasto.

Consultar todas las divisiones asociadas a un viaje a través de sus gastos.

Calcular montos pendientes por participante.

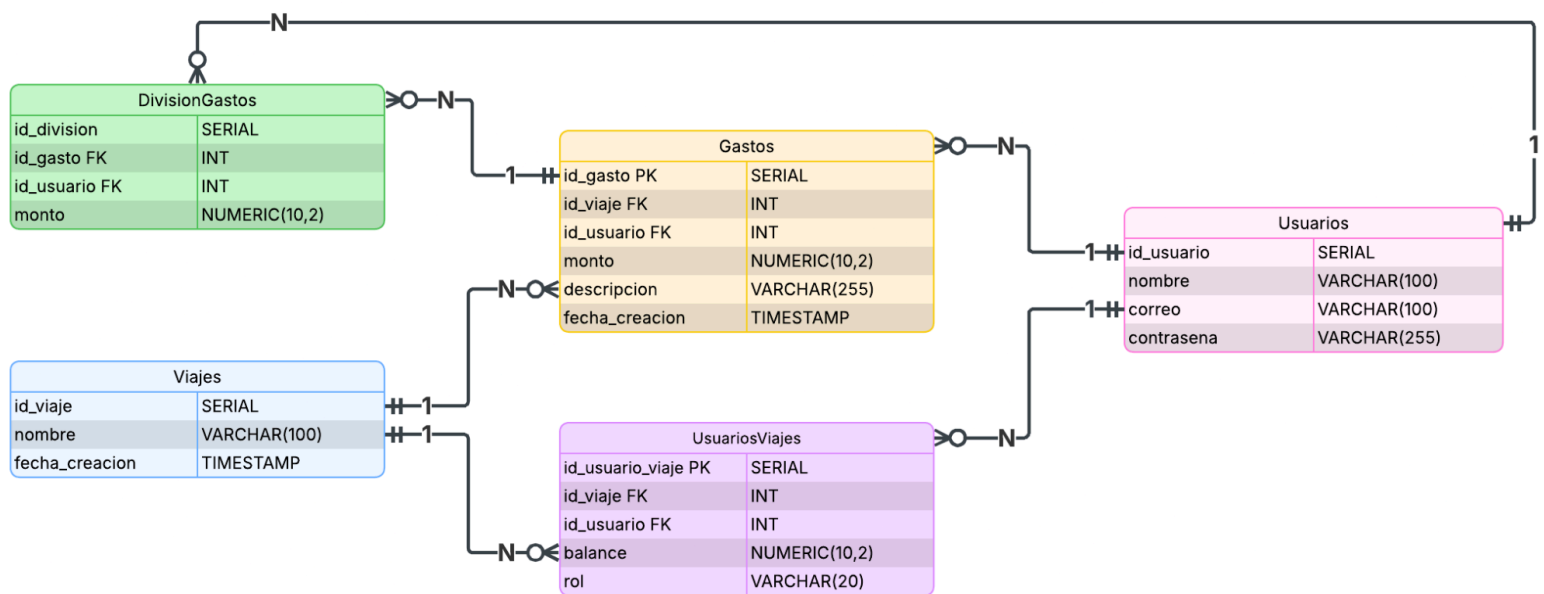


Figura 1. Diagrama Entidad–Relación de TravelSplit con cardinalidades.

4. Análisis de tamaños

En esta sección se presenta el análisis de tamaños del modelo de datos, con el objetivo de estimar el espacio de almacenamiento requerido y evaluar la eficiencia del diseño propuesto. Este análisis permite comprender el impacto que tiene la elección de tipos de datos y la estructura de las tablas en el rendimiento y escalabilidad de la base de datos.

4.1 Tamaño de registro

El análisis del tamaño de registro permite estimar cuánto espacio ocupa cada fila dentro de una tabla. Esto resulta útil para evaluar la eficiencia del modelo, prever el crecimiento de almacenamiento y justificar la selección de tipos de datos. En este análisis se toma como referencia la estructura definida para las tablas Usuarios, Viajes, UsuariosViajes, Gastos y DivisionGastos, así como los tipos de datos asignados en el modelo propuesto

Para efectos de estimación, se utilizarán los siguientes tamaños aproximados:

SERIAL: 4 bytes

INT: 4 bytes

TIMESTAMP: 8 bytes

NUMERIC(10,2): 8 bytes

VARCHAR(n): se considera el tamaño máximo definido.

4.1.1 Usuarios

Tamaño estimado por columna:

id_usuario - 4 bytes

nombre - 64 bytes

correo - 64 bytes

contrasena - 256 bytes

Tamaño estimado por registro: 388 bytes

Este tamaño es razonable, ya que concentra principalmente datos de identificación y autenticación. El campo de mayor peso es contrasena, debido a que se reserva suficiente espacio para almacenar hashes seguros.

4.1.2 Viajes

Tamaño estimado por columna:

id_viaje - 4 bytes

nombre - 64 bytes

fecha_creacion - 8 bytes

Tamaño estimado por registro: 76 bytes

Esta tabla presenta un tamaño bajo, lo que la hace eficiente en almacenamiento y consultas.

4.1.3 UsuariosViajes

Tamaño estimado por columna:

id_usuario_viaje - 4 bytes

id_viaje - 4 bytes

id_usuario - 4 bytes

balance - 8 bytes

rol - 16 bytes

Tamaño estimado por registro: 36 bytes

Esta tabla tiene un tamaño pequeño y eficiente. Su diseño es adecuado para una tabla intermedia de relación muchos a muchos.

4.1.4 Gastos

Tamaño estimado por columna:

id_gasto → 4 bytes

id_viaje → 4 bytes

id_usuario → 4 bytes

monto → 8 bytes

descripcion → 256 bytes

fecha_creacion → 8 bytes

Tamaño estimado por registro: 284 bytes

Esta es una de las tablas más relevantes del sistema, ya que concentra la operación principal del dominio. El mayor peso lo aporta descripción, pero se justifica porque permite registrar información clara sobre cada gasto.

4.1.5 DivisionGastos

Tamaño estimado por columna:

id_division- 4 bytes

id_gasto- 4 bytes

id_usuario - 4 bytes

monto - 8 bytes

Tamaño estimado por registro: 20 bytes

Es la tabla más liviana del modelo, lo cual resulta conveniente porque puede crecer rápidamente cuando un gasto se divide entre múltiples participantes.

4.2 Análisis del tamaño de tabla

El análisis del tamaño de tabla permite estimar cuánto espacio total podría ocupar cada entidad conforme el sistema crezca. Dado que TravelSplit está orientado a viajes grupales y división de gastos, se proyecta un crecimiento moderado, especialmente en las tablas relacionadas con gastos y divisiones.

Para este análisis se propone un escenario de crecimiento inicial realista/optimista:

10,000 usuarios registrados

5,000 viajes creados

promedio de 4 participantes por viaje

promedio de 8 gastos por viaje

promedio de 4 divisiones por gasto

Estas cifras permiten visualizar el comportamiento del modelo en una primera etapa de uso sostenido.

4.2.1 Usuarios

Tamaño por registro: 388 bytes

Proyección: 10,000 registros

Cálculo:

$$388 \times 10,000 = 3,880,000 \text{ bytes}$$

Tamaño estimado total: 3.88 MB

La tabla Usuarios tendría un tamaño manejable incluso con varios miles de registros.

4.2.2 Viajes

Tamaño por registro: 76 bytes

Proyección: 5,000 registros

Cálculo:

$$76 \times 5,000 = 380,000 \text{ bytes}$$

Tamaño estimado total: 0.38 MB

La tabla Viajes mantiene un tamaño bajo, por lo que su crecimiento no representa una carga importante para la base de datos.

4.2.3 UsuariosViajes

Si se estima un promedio de 4 participantes por viaje y existen 5,000 viajes:

$$5,000 \times 4 = 20,000 \text{ registros}$$

Tamaño por registro: 36 bytes

Proyección: 20,000 registros

Cálculo:

$$36 \times 20,000 = 720,000 \text{ bytes}$$

Tamaño estimado total: 0.72 MB

A pesar de su mayor volumen de registros, sigue siendo una tabla eficiente por su tamaño reducido.

4.2.4 Gastos

Si se estima un promedio de 8 gastos por viaje y existen 5,000 viajes:

$$5,000 \times 8 = 40,000 \text{ registros}$$

Tamaño por registro: 284 bytes

Proyección: 40,000 registros

Cálculo:

$$284 \times 40,000 = 11,360,000 \text{ bytes}$$

Tamaño estimado total: 11.36 MB

Esta tabla tendría uno de los mayores tamaños del modelo, lo cual es lógico porque almacena la actividad principal del sistema.

4.2.5 DivisionGastos

Si cada gasto se divide, en promedio, entre 4 participantes, y existen 40,000 gastos:

$$40,000 \times 4 = 160,000 \text{ registros}$$

Tamaño por registro: 20 bytes

Proyección: 160,000 registros

Cálculo:

$$20 \times 160,000 = 3,200,000 \text{ bytes}$$

Tamaño estimado total: 3.20 MB

Esta tabla podría crecer rápidamente en cantidad de registros, pero su tamaño por fila es muy pequeño, por lo que sigue siendo eficiente.

4.3 Resumen general del almacenamiento proyectado

A partir de las estimaciones anteriores, el almacenamiento total aproximado del modelo sería el siguiente:

Usuarios - 3.88 MB

Viajes - 0.38 MB

UsuariosViajesn- 0.72 MB

Gastos - 11.36 MB

DivisionGastos - 3.20 MB

Tamaño total estimado del modelo: 19,54 MB

Este resultado muestra que el modelo es liviano y escalable para una primera etapa de operación. Incluso con miles de usuarios, viajes y gastos, el espacio requerido sigue siendo relativamente bajo, lo que demuestra que la selección de tipos de datos es adecuada y eficiente con respecto al crecimiento esperado del sistema

5. Conclusiones

El modelo relacional propuesto para TravelSplit permite representar de forma clara y estructurada la información necesaria para gestionar gastos compartidos en viajes grupales. La organización alrededor de las entidades Usuarios, Viajes, UsuariosViajes, Gastos y DivisionGastos refleja adecuadamente las relaciones principales del sistema y facilita su futura implementación en una base de datos relacional

Además, la definición de estándares de nombrado, llaves primarias y foráneas, restricciones de integridad y tipos de datos contribuye a mantener consistencia, legibilidad y control sobre la información almacenada. Esto fortalece la calidad del diseño y permite reducir errores, duplicidades y problemas de integridad en la base de datos

A partir del análisis de tamaño de registro y de proyección de crecimiento, se concluye que el modelo es eficiente en términos de almacenamiento. Las tablas más voluminosas, como Gastos y DivisionGastos, mantienen un tamaño razonable incluso ante escenarios de uso con miles de registros, lo cual evidencia que la estructura propuesta posee un buen nivel de escalabilidad para una primera versión del sistema.

En conclusión, el diseño presentado cumple con los objetivos de claridad, integridad y eficiencia planteados para TravelSplit, y constituye una base sólida para el desarrollo posterior de la aplicación.